LOC3X1 Modernization PoC Handbook

Alberto Hernandez

November 3, 2025

Abstract

This handbook explains, from first principles, the technologies and design behind a modernization Proof of Concept (PoC) for the LOC3X1 domain. It covers the client-provided `Dependencies` folder (WSDL/XSD, mapping specs, BPEL), the modern Spring Boot + Apache Camel implementation, and how mapping, validation, and enrichment combine to produce the final XML output. The content is intended for readers with no prior knowledge of Java, Maven, SOAP, or BPEL, and includes diagrams, tables, and pseudocode to make the logic approachable. References are provided and are clickable.

Contents

1 Background and Goals

1.1 Problem Context

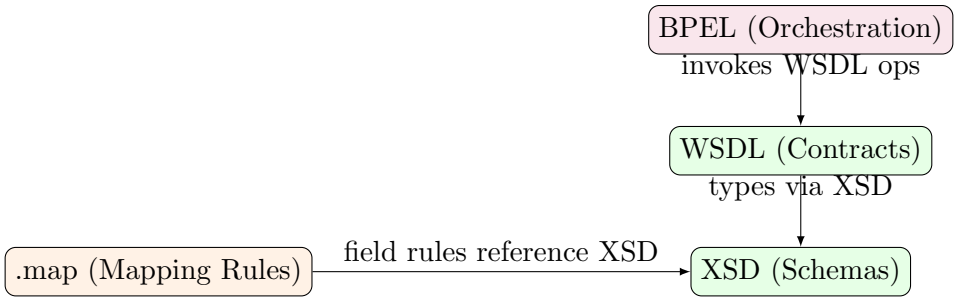
The client supplied an authoritative `Dependencies` folder containing: WSDL (service contracts), XSD (data schemas), mapping specifications (`.map`), and a BPEL process definition. The modernisation goal is to implement equivalent business logic in a cloud-friendly, maintainable stack, demonstrating correctness by producing deterministic XML output.

This modernisation effort addresses a fundamental challenge in enterprise software evolution: legacy systems built on vendor-specific orchestration engines and proprietary tooling must be translated into open, portable frameworks that support contemporary development practices—version control, automated testing, continuous integration, and containerised deployment. The client's `Dependencies` folder represents the complete specification of a location retrieval workflow originally implemented using early-2000s enterprise service bus (ESB) technology. Our task is to preserve the exact business semantics whilst replacing the execution substrate.

1.2 Level 1: Client Legacy Landscape (Old Technology)

This first level introduces the original code and specifications living in `CHUBB/Dependencies/`. It explains how contracts (WSDL), schemas (XSD), mapping specifications (`.map`), and BPEL orchestrations fit together. Understanding these artefacts is essential: they encode not merely technical details but the business logic, data governance rules, and integration contracts that have been validated over years of production use.

Legacy-only Diagram



Key Concepts for Modern Developers Before examining the code samples, it is crucial to understand what each technology accomplishes and why it exists. These are not arbitrary choices; each addresses a specific problem in distributed enterprise systems.

- **WSDL (Web Services Description Language)**: A WSDL document functions as a formal contract between service provider and consumer. It answers three essential questions: *What operations are available?* (e.g., `GetLocationList`), *What data structures do they accept and return?* (messages with typed parts), and *How do I communicate?* (protocol bindings, typically SOAP over HTTP). If you have encountered OpenAPI (formerly Swagger) specifications for RESTful APIs, WSDL serves an analogous purpose for SOAP-based web services: it provides machine-readable metadata enabling automated client generation, compile-time validation, and runtime introspection. The WSDL vocabulary includes `types` (data schemas), `messages` (operation parameters), `portType` (abstract operations), `binding` (concrete protocol mapping), and `service` (network endpoints). For a modern developer, understanding WSDL is essential because many enterprise systems still expose SOAP interfaces, and tools like Apache CXF or JAX-WS can generate Java or Python client code directly from WSDL, eliminating manual serialisation logic.

  Step 1: Generate stubs from WSDL (wsdl2java/wsimport)
  Implemented: WSDLs in `CHUBB/Dependencies/LocationRetrievalLOC3X1*/` produce client stubs packaged in `libs/soap-clients-0.0.1-SNAPSHOT.jar`; these types are referenced by routes/controllers (e.g., demo client at `GET /api/index`).
  Enhance: Move stub generation into the build (Maven/Gradle) with locked plugin versions, generate sources to `target/generated-sources`, and include smoke tests that call mocked SOAP endpoints to ensure stubs align with WSDL changes.

- **XSD (XML Schema Definition)**: An XSD file defines the permissible structure, data types, constraints, and cardinality of XML documents. Whereas WSDL describes *what you can do*, XSD describes *what shape the data must have*. Think of XSD as a formal grammar or type system for XML: it specifies that a `StatusInformation` element must contain exactly one `StatusCode` (a string) and zero or one `StatusMessage` elements, enforced via `minOccurs` and `maxOccurs` attributes. XSD supports complex types (structured elements with child nodes), simple types (atomic values with restrictions like patterns or enumerations), and inheritance-like mechanisms (extension and restriction). Schema validation occurs before business logic executes, ensuring malformed or incomplete data never enters the system. From a modern perspective, XSD is analogous to JSON Schema or Protocol Buffers: it provides a contract that tooling can enforce, enabling automated validation, code generation (e.g., JAXB in Java, `xsData` in Python), and documentation. When you see an XSD, you are looking at the authoritative definition of message formats—deviating from it will cause integration failures.

Step 2: Locate and govern XSDs (Schemas)
Implemented: XSDs live under `CHUBB/Dependencies/Schemas/`; JAXB-derived POJOs are provided by `libs/xsd-javapojos-0.0.3-SNAPSHOT.jar` and used across validation/mapping.
Enhance: Establish a schema registry (project-level ownership, versioning), generate POJOs via build plugins (`maven-jaxb2-plugin`/`cxf-codegen-plugin`) with version pinning, and add CI checks that validate all XML artefacts against authoritative XSDs. Maintain a coverage table linking XSD elements to consumers (routes, mappers).

- **.map Specifications**: The client's `.map` files are tabular specifications documenting field-level transformation rules: "Copy `Address/City` from the incoming message to `Location/Municipality` in the outgoing message," or "If `StatusCode` is `SUCCESS`, map it to `0`; otherwise map to `1`." These specifications originate from proprietary mapping tools (e.g., Oracle ESB Mapper, IBM WebSphere Transformation Extender) where business analysts could define transformations visually or in spreadsheets without writing code. However, `.map` files themselves are not executable—they are documentation. To operationalise them, we translate every row into XSLT code (or equivalent logic), which becomes part of the version-controlled codebase. This translation is critical: XSLT is a standard, testable, and portable transformation language, whereas `.map` files are vendor-specific. For a novice developer, the key insight is that `.map` files encode *business knowledge*—which fields matter, which transformations are legally required, which defaults apply—and this knowledge must be faithfully preserved during modernisation.

Step 3: Translate .map specifications into XSL/XSLT
Implemented: `CHUBB/main_mapper_with_submaps.xsl` encodes field-level rules from `.map` specifications and is applied in the mapping stage.
Enhance: Maintain a mapping coverage matrix (each `.map` row → XSLT template), add golden-file tests for inputs/outputs, enable XSLT 2.0 features where helpful, and log mapping decisions for auditability.

- **BPEL (Business Process Execution Language)**: BPEL is an XML-based language for orchestrating web service invocations into workflows. A BPEL process reads like an imperative programme: "Receive a request from the client, invoke Service A with certain parameters, wait for its response, then conditionally invoke Service B or Service C based on the outcome, finally merge results and reply." BPEL supports constructs familiar to programmers—sequence, conditional branches (`if`), loops (`while`, `forEach`), parallel execution (`flow`), exception handling (`catch`)—but expressed in verbose XML rather than a general-purpose language. BPEL processes execute within a BPEL engine (e.g., Oracle BPEL Process Manager, IBM WebSphere Process Server), which manages state, transactions, and long-running interactions. For modernisation, we extract the control flow and business logic from the BPEL XML and re-implement it as a Camel route in YAML: the *semantics* remain identical (same sequence of calls, same conditionals), but the execution substrate changes from a proprietary engine to an open-source integration framework. Understanding BPEL is essential because it documents the *intended behaviour*—the BPEL file is the canonical source of truth for "what should happen when."

Step 4: Evaluate BPEL/Mediate files and translate to routes
Implemented: `CHUBB/Dependencies/LocationRetrievalLOC3X1Process.bpel` is the orchestration-of-record; its sequence and decisions are re-expressed as Camel YAML routes.
Enhance: Keep a spec-of-record document that explicitly maps BPEL constructs to route steps, include parity tests (legacy vs. modern outputs), and document any divergence with justification.

Legacy Understanding (Pseudocode) Before diving into code samples, Algorithm 1 formalises the process of ingesting and interpreting the client's legacy assets. This is not merely a file-reading exercise; it represents the reverse-engineering phase where we reconstruct the business logic, data contracts, and integration points from static artefacts.

Algorithm 1 Understand Legacy Assets

1: **function** READLEGACYSPECS(depsFolder)
2: wsdl ← loadContracts(depsFolder/`LocationRetrievalLOC3X1M`)
3: xsd ← loadSchemas(depsFolder/`Schemas`)
4: map ← loadMapSpecs(depsFolder/`.map`)
5: bpel ← loadOrchestration(depsFolder/`LocationRetrievalLOC3X1Process.bpel`)
6: explain(wsdl, xsd, map, bpel) ▷ produce human-friendly notes and diagrams
7: **end function**

Line-by-line interpretation:

- **Line 2**: `loadContracts` parses WSDL files to extract operation signatures, message structures, and binding protocols. This step identifies *which services* the workflow invokes and *what data* they require.
- **Line 3**: `loadSchemas` ingests XSD files, building a type registry that validates all XML payloads. This ensures we know the precise shape of every data structure.
- **Line 4**: `loadMapSpecs` reads transformation rules, which will later be translated into XSLT. This captures *how* data flows from one schema to another.
- **Line 5**: `loadOrchestration` parses BPEL to reconstruct the workflow sequence: validation, enrichment, external calls, and response assembly.
- **Line 6**: `explain` synthesises these artefacts into developer documentation—the handbook you are reading—complete with diagrams and pseudocode that make implicit semantics explicit.

This algorithm underscores a key principle: *modernisation is not rewriting from scratch; it is faithful translation guided by authoritative specifications.*

Legacy Code Samples (Simplified Excerpts) Below are small, simplified snippets representative of the client-provided assets in `CHUBB/Dependencies/`. Each snippet is paired with a detailed explanation to ground a modern developer in the legacy semantics. These examples are not toy demonstrations—they mirror real enterprise artefacts, and understanding them is essential for correct modernisation.

WSDL: LocationRetrievalLOC3X1M (operations/messages)

Listing 1: WSDL operation wiring input/output messages

```
<wsdl:definitions targetNamespace="http://example.com/loc3x1m"
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <wsdl:types>
    <xsd:schema targetNamespace="http://example.com/loc3x1m/types">
      <!-- XSD types imported/referenced here -->
    </xsd:schema>
  </wsdl:types>
  <wsdl:message name="GetLocationListRequest">
    <wsdl:part name="parameters" element="xsd:GetLocationList"/>
  </wsdl:message>
  <wsdl:message name="GetLocationListResponse">
    <wsdl:part name="parameters" element="xsd:GetLocationListResult"/>
  </wsdl:message>
  <wsdl:portType name="LocationRetrieval">
    <wsdl:operation name="GetLocationList">
      <wsdl:input message="tns:GetLocationListRequest"/>
      <wsdl:output message="tns:GetLocationListResponse"/>
    </wsdl:operation>
  </wsdl:portType>
</wsdl:definitions>
```

High-level explanation: Listing 1 defines a SOAP web service contract for location retrieval. The WSDL specifies one operation, `GetLocationList`, which accepts a request message and returns a response message. Both messages reference XSD-defined elements, ensuring that any client invoking this service must conform to the schema. This contract enables automated client generation—tools like Apache CXF's `wsdl2java` or Python's `zeep` can read this WSDL and generate type-safe proxy classes.

Detailed line-by-line breakdown:

- **Lines 1–3**: The `<wsdl:definitions>` root element establishes the document's target namespace (`http://example.com/loc3x1m`) and declares necessary XML namespaces. The `targetNamespace` uniquely identifies this service contract; other systems reference operations via this namespace. The `xmlns:wsdl` and `xmlns:xsd` declarations import the WSDL and XML Schema vocabularies, respectively. These lines are boilerplate but essential—omitting them or using incorrect namespace URIs breaks tooling.

- **Lines 4–8**: The `<wsdl:types>` section embeds or imports XSD schema definitions. Line 5 opens a schema declaration with its own target namespace (`http://example.com/loc3x1m/types`), which will house custom types like `GetLocationList` and `GetLocationListResult`. The comment on line 6 indicates that actual type definitions are imported (via `<xsd:import>`) or defined inline. In production WSDLs, this section can span hundreds of lines, defining dozens of complex types. Understanding that `<wsdl:types>` *is* XSD content is crucial: WSDL does not invent a new type system; it reuses XML Schema.

- **Lines 9–11**: The `<wsdl:message>` element named `GetLocationListRequest` declares the structure of the input message. Line 10 specifies that this message has one `<wsdl:part>` named `parameters`, whose content is defined by the XSD element `xsd:GetLocationList` (from the `<wsdl:types>` section). The term "part" is historical: SOAP originally allowed messages to be split into multiple parts (headers, body fragments), though modern practice uses a single document-style part. For a developer, the key takeaway is that `element="xsd:GetLocationList`

creates a strong coupling: the request payload must validate against the `GetLocationList` XSD element, ensuring type safety.

- **Lines 12–14**: Similarly, `GetLocationListResponse` defines the output message structure. Line 13 binds the response to the `xsd:GetLocationListResult` element. This symmetry—request and response both referencing XSD elements—enables bidirectional validation: the client validates the request before sending, and the server validates the response before replying.

- **Lines 15–20**: The `<wsdl:portType>` element named `LocationRetrieval` declares the abstract interface—analogous to a Java interface or Python abstract base class. Line 16 defines the `GetLocationList` operation. Lines 17–18 wire this operation to the previously declared messages: `<wsdl:input>` references `GetLocationListRequest` (via the `tns:` prefix, meaning "target namespace"), and `<wsdl:output>` references `GetLocationListResponse`. This establishes a request-response pattern: invoke `GetLocationList` with a `GetLocationListRequest`, receive a `GetLocationListResponse`. Other patterns exist (one-way, notification), but request-response dominates enterprise SOAP. Line 20 closes the `portType`.

- **Line 21**: Closes the WSDL document. In a complete WSDL, subsequent sections (`<wsdl:binding>`, `<wsdl:service>`) would specify protocol details (SOAP 1.1 vs. 1.2, HTTP vs. JMS) and endpoint URLs, but those are omitted here for brevity.

Integration insight: When the BPEL process (Listing 3, line 5) invokes `LOC3X1M.GetLocationList`, it relies on this WSDL to know what data structure to send (`GetLocationListRequest`) and what to expect back (`GetLocationListResponse`). The BPEL engine reads the WSDL at deployment time, generates internal proxies, and validates messages at runtime. Our modern Camel implementation must replicate this: generate Java POJOs from the XSD, construct a `GetLocationListRequest` object, invoke the SOAP service (using CXF or JAX-WS), and parse the `GetLocationListResponse`.

XSD: StatusInformation.xsd (structure and types)

Listing 2: XSD element declaring shape and types

```
1  <xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
2              targetNamespace="http://example.com/common">
3    <xsd:element name="StatusInformation" type="common:StatusInformationType"/>
4    <xsd:complexType name="StatusInformationType">
5      <xsd:sequence>
6        <xsd:element name="StatusCode" type="xsd:string"/>
7        <xsd:element name="StatusMessage" type="xsd:string" minOccurs="0"/>
8      </xsd:sequence>
9    </xsd:complexType>
10 </xsd:schema>
```

High-level explanation: Listing 2 defines a reusable data structure, `StatusInformation`, which appears in responses from multiple services. This schema encodes mandatory and optional fields, data types, and ordering constraints. XSD serves as the single source of truth for message structure: code generators produce classes that mirror this schema exactly, ensuring runtime data conforms to design-time contracts.
Detailed line-by-line breakdown:

- **Lines 1–2**: The `<xsd:schema>` root element establishes the schema's target namespace (`http://example.com/common`). Any element or type defined here belongs to this names-

pace, and other schemas import it via `<xsd:import>`. The `xmlns:xsd` declaration imports the XML Schema vocabulary itself, enabling use of constructs like `<xsd:element>` and `<xsd:complexType>`. Understanding namespaces is critical in enterprise XML: they prevent name collisions (multiple schemas might define a `StatusInformation` element) and establish ownership (the `common` namespace might be governed by a central architecture team).

- **Line 3**: Declares a global element named `StatusInformation`, typed as `common:StatusInformationType`. The `common:` prefix refers to the `targetNamespace` declared on line 2. Global elements are entry points: they can appear as top-level elements in XML documents or be referenced by WSDL messages (as in Listing 1, line 10). The separation of element and type (rather than defining structure inline) enables reuse: other elements could also be typed as `StatusInformationType`, or this type could be extended or restricted elsewhere.

- **Line 4**: Defines `StatusInformationType` as a `<xsd:complexType>`, meaning it contains child elements (as opposed to `<xsd:simpleType>`, which restricts atomic values like strings or integers). Complex types model structured data—objects in object-oriented parlance.

- **Line 5**: The `<xsd:sequence>` compositor dictates that child elements must appear *in the order specified*. XML Schema supports other compositors (`<xsd:choice>` for alternatives, `<xsd:all>` for unordered sets), but `<xsd:sequence>` dominates in enterprise messaging because strict ordering simplifies parsing and generation. This means `StatusCode` must always precede `StatusMessage` in serialised XML.

- **Line 6**: Declares `StatusCode` as a required child element of type `xsd:string`. The absence of `minOccurs` or `maxOccurs` attributes implies defaults: `minOccurs="1"` and `maxOccurs="1"`, meaning exactly one `StatusCode` must be present. Validators will reject documents lacking this element. The `xsd:string` type is a built-in simple type allowing any Unicode string content. In production schemas, this might be restricted further (e.g., pattern matching `[0-9]3` for HTTP-style status codes, or an enumeration like `SUCCESS | FAILURE | PENDING`).

- **Line 7**: Declares `StatusMessage` as an optional child element, signalled by `minOccurs="0"`. The implicit `maxOccurs="1"` means at most one `StatusMessage` may appear. Optional fields are ubiquitous in real-world schemas: they model scenarios where supplementary information (like a human-readable error description) may or may not be available. Code generators typically map optional elements to nullable types (`String?` in Kotlin, `Optional<String>` in Java, `Optional[str]` in Python). Handling optionality correctly is critical: business logic must not assume `StatusMessage` is present.

- **Lines 8–9**: Close the `<xsd:sequence>` and `<xsd:complexType>`.

- **Line 10**: Closes the schema document.

Integration insight: The WSDL messages (Listing 1) reference elements like `GetLocationListResult`, which internally contain `StatusInformation` as a nested field. When the BPEL process (Listing 3, line 5) receives a response from `LOC3X1M`, it validates the response against the schema, ensuring `StatusCode` is present and `StatusMessage` adheres to expected types. Our mapping logic (Section 5) must correctly handle optional fields: if `StatusMessage` is absent in the LOC3X1M reply, the XSLT transformation must not produce an empty or malformed element in the LOC3X1B output. XSD governs data integrity end-to-end.

BPEL: LocationRetrievalLOC3X1Process.bpel (orchestration)

Listing 3: BPEL recipe sequencing a service call

```
<process name="LocationRetrievalLOC3X1Process"
  xmlns="http://docs.oasis-open.org/wsbpel/2.0/process/executable">
  <sequence>
    <receive partnerLink="Client" operation="Start"/>
    <invoke partnerLink="LOC3X1M" operation="GetLocationList" inputVariable="req" outputVariable="
        res"/>
    <reply partnerLink="Client" operation="Done" variable="res"/>
  </sequence>
</process>
```

High-level explanation: Listing 3 defines an executable business process that orchestrates a synchronous service call. This BPEL process accepts an inbound request, invokes the `GetLocationList` operation on the `LOC3X1M` service (defined in Listing 1), and returns the result to the caller. The BPEL engine manages execution state, timeouts, and error handling (though error handling constructs are omitted here for brevity). Our modernisation replaces this BPEL process with a Camel route that implements identical control flow.

Detailed line-by-line breakdown:

- **Lines 1–2**: The `<process>` root element declares this as an executable BPEL process named `LocationRetrievalLOC3X1Process`. The `xmlns` attribute establishes the BPEL 2.0 namespace, adhering to the OASIS WS-BPEL specification. BPEL 2.0 introduced significant improvements over BPEL 1.1 (clearer scoping rules, richer error handling, standardised syntax), and the `executable` designation indicates this process is meant to run—opposed to `abstract` processes, which serve as documentation or templates.

- **Line 3**: The `<sequence>` activity specifies that enclosed activities execute sequentially, in order. BPEL supports parallel execution via `<flow>`, conditional branching via `<if>`, and iteration via `<forEach>` and `<while>`, but this simple process requires only a linear sequence. Understanding `<sequence>` is essential: each step completes before the next begins, and failure at any step typically aborts the process (unless caught by a fault handler).

- **Line 4**: The `<receive>` activity waits for an inbound message from the `Client` partner link, specifically for the `Start` operation. A **partner link** is BPEL's abstraction for an external entity: it references a WSDL port type and binding, establishing how the process communicates with that entity. The `Client` partner link represents the caller who initiated this workflow. The `operation="Start"` corresponds to a WSDL operation defined in the process's own service interface (not shown here). Upon receiving the message, the BPEL engine stores it in an implicit variable for subsequent use (or an explicitly named variable if `variable="..."` were present). This line is the process entry point.

- **Line 5**: The `<invoke>` activity calls an external web service. Specifically, it invokes the `GetLocationList` operation on the `LOC3X1M` partner link. The `LOC3X1M` partner link is configured (in deployment descriptors not shown here) to point to the WSDL from Listing 1, establishing the contract for this invocation. The `inputVariable="req"` attribute specifies that the request payload is held in a process variable named `req` (which was populated either by the `<receive>` or via an `<assign>` activity not shown). The `outputVariable="res"` attribute directs the engine to store the response in a variable named `res`. This line represents the core business logic: query the location service. The BPEL engine handles SOAP serialisation, HTTP transport, and response parsing transparently—developers declaratively specify *what* to invoke, not *how* to marshal data. Our modern implementation must replicate this behaviour using Camel's `to("cxf:...")` or `to("http:...")` components.

- **Line 6**: The `<reply>` activity sends a response back to the `Client` partner link via the `Done` operation, using the `res` variable as the payload. This completes the request-response interaction initiated by line 4. After replying, the process instance terminates successfully (assuming no outstanding parallel flows or event handlers). The `Done` operation must be defined in the process's own WSDL interface, ensuring the client knows what message structure to expect.
- **Lines 7–8**: Close the `<sequence>` and `<process>`.

Integration insight: This BPEL process encapsulates the workflow's control flow. In a complete system, additional `<invoke>` activities would appear for enrichment services (`CRP11X1` for state lookup, `CRP10X1` for country lookup), and `<assign>` activities would manipulate variables (applying defaults, transforming data structures). The orchestration documented here is simplified, but the principle holds: BPEL sequences service calls, and our Camel route must mirror this sequence exactly. If the BPEL invokes Service A, then Service B, then Service C, the Camel route must invoke them in the same order with the same conditional logic. BPEL is the blueprint; our code must be a faithful implementation.

Connection and Logic Across Code Boxes Understanding how these three artefacts interlock is essential for correct modernisation:

- **WSDL messages reference XSD elements**: In Listing 1, lines 9–14 define `GetLocationListRequest` and `GetLocationListResponse` messages. Lines 10 and 13 explicitly bind these messages to XSD elements (`xsd:GetLocationList` and `xsd:GetLocationListResult`). This creates a strong dependency: the WSDL cannot function without the corresponding XSD definitions in the `<wsdl:types>` section (lines 4–8). When the BPEL process invokes the service, the request payload must conform to the `GetLocationList` schema, and the response will conform to the `GetLocationListResult` schema. Code generators exploit this binding: running `wsdl2java` on Listing 1 produces Java classes whose fields mirror the XSD types exactly, ensuring compile-time type safety.
- **BPEL invocations target WSDL operations**: In Listing 3, line 5 invokes the `GetLocationList` operation on partner link `LOC3X1M`. This operation is defined in Listing 1, lines 16–19, which specify input and output messages. The BPEL engine resolves `LOC3X1M` to a WSDL endpoint (via deployment configuration), reads the WSDL to determine message formats, serialises the `req` variable into a SOAP envelope conforming to `GetLocationListRequest`, transmits it over HTTP, deserialises the response into the `res` variable (conforming to `GetLocationListResponse`), and validates both payloads against their schemas. If the response violates the schema—say, `StatusCode` is missing—the engine raises a fault, which BPEL can catch and handle. Our Camel implementation must replicate this end-to-end flow: build the request, invoke the service, validate the response, and handle errors.

Step 5: Wire WSDL stubs into SOAP clients (runtime integration)
Implemented: SOAP clients for `LOC3X1M`, `CRP11X1`, and `CRP10X1` are invoked from the Spring Boot application and Camel routes; endpoints are configurable via `application.yml`.
Enhance: Use Apache CXF bean configuration for timeouts, WS-Security, and consistent error handling; add WireMock-based integration tests; enforce retries/backoff at the route level; and publish client metrics (latency, error rates) via Actuator.

- **Optional fields propagate through the entire pipeline**: In Listing 2, line 7 declares `StatusMessage` as optional (`minOccurs="0"`). This seemingly minor detail has profound implications. When this element appears within `GetLocationListResult` (referenced by Listing 1, line 13), the WSDL contract permits responses where `StatusMessage` is absent. The BPEL process (Listing 3, line 5) stores such responses in the `res` variable, which may or may not contain `StatusMessage`. Subsequently, when mapping logic transforms the LOC3X1M response to the LOC3X1B format (Section 5), the XSLT or Java code must test for `StatusMessage` presence before attempting to copy it. Failing to handle optionality correctly leads to null pointer exceptions, malformed XML, or silent data loss. This illustrates why XSD is authoritative: optional fields in the schema mandate defensive coding throughout the entire stack.

- **Cross-cutting validation ensures data integrity**: The interplay of these artefacts creates multiple validation checkpoints. (1) When the client sends a request to the BPEL process (Listing 3, line 4), the engine validates it against the process's input schema. (2) Before invoking `LOC3X1M` (line 5), the engine validates the `req` variable against `GetLocationListRequest` (Listing 1, line 10). (3) The `LOC3X1M` service itself validates the incoming SOAP envelope against its schema. (4) The service's response is validated against `GetLocationListResponse` (Listing 1, line 13) before being stored in `res`. (5) Finally, the BPEL reply (line 6) is validated against the process's output schema. This defence-in-depth strategy ensures malformed data never propagates. Our modern implementation must preserve these validation stages, using JAXB unmarshalling (which throws exceptions on schema violations) and explicit validator invocations where necessary.

Why this matters for modernisation: A novice might assume, "I'll just rewrite the logic in Java or Python without worrying about WSDL/XSD/BPEL." This approach is catastrophic. The legacy artefacts encode business rules, error-handling semantics, and data constraints validated over years of production use. Ignoring them invites subtle bugs: missing validation, incorrect field mappings, mishandled optionals. Correct modernisation *interprets* these artefacts—generating code from WSDL/XSD (ensuring type safety), translating BPEL control flow into Camel routes (preserving orchestration semantics), and converting `.map` rules into XSLT (maintaining transformation correctness). The goal is not to discard legacy knowledge but to express it in a modern, maintainable form.

1.3 Level 2: Combined Flow (Legacy + Modern Translation)

This second level shows how the original specifications drive the modern translation pipeline (Spring Boot + Apache Camel), resulting in a deterministic XML artefact. Whereas Subsection 1.2 focused on understanding the legacy artefacts in isolation, this section illustrates their *operational integration*—how they collectively define a data processing workflow, and how that workflow is re-implemented using contemporary technologies.

Step 6: Spring Boot scaffolding (runtime, config, health)
Implemented: The project is a Spring Boot app with Camel YAML routes, externalised configuration in `src/main/resources/application.yml`, and controllers/services wired under `com.locationretrievalloc3x1process.webservice.*`.
Enhance: Add Actuator health/metrics, centralise configuration (profiles/secrets), apply structured logging, and document operational runbooks (deploy, observe, roll back).

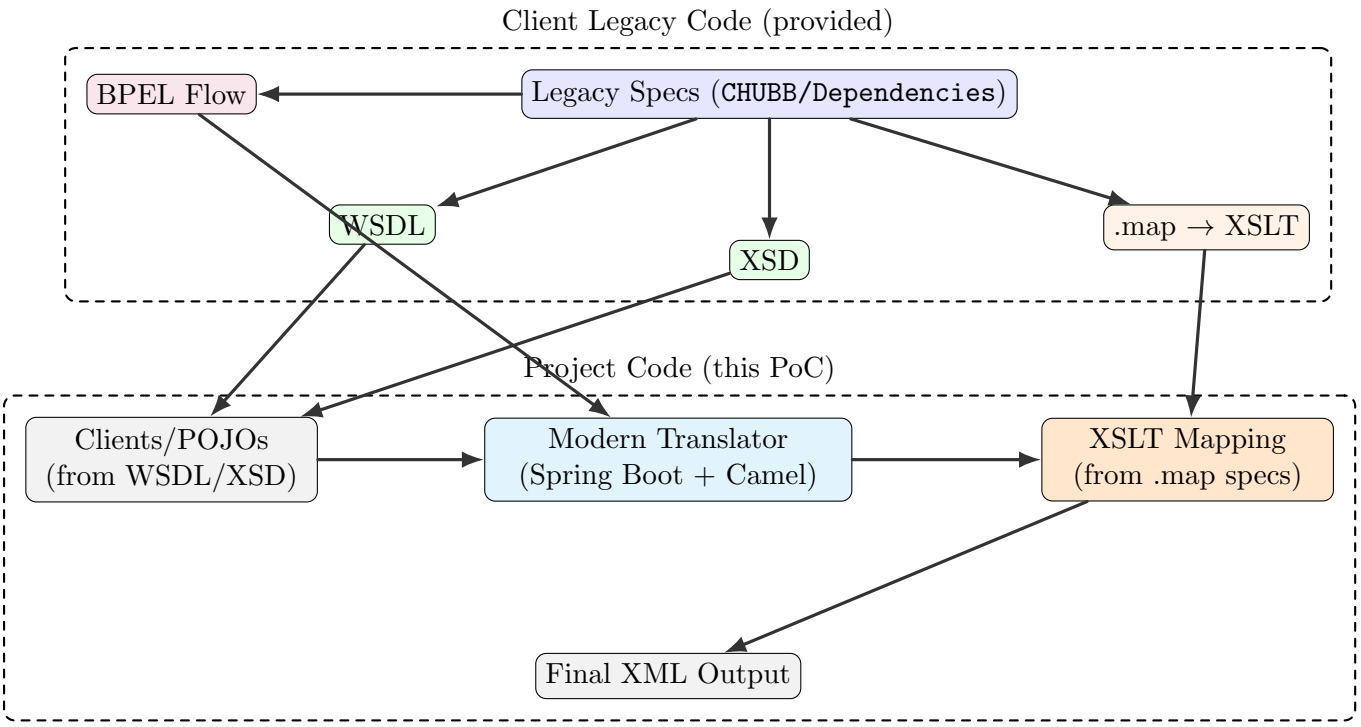


Figure 1: Legacy-to-modern translation pipeline. Client artefacts (top cluster) feed the PoC (bottom cluster): WSDL/XSD drive POJO generation via `wsdl2java/xjc`; BPEL flow is manually translated into Camel route logic; `.map` specifications become XSLT stylesheets invoked by the route to transform LOC3X1M responses into LOC3X1B format; final XML is written to `CHUBB/output/loc3x1_result.xml` for regression testing.

Combined Diagram

Translation Workflow (Pseudocode) Algorithm 2 formalises the end-to-end translation and execution process. This is not merely pseudocode for pedagogical purposes—it directly corresponds to the Camel route implementation described in Section 4.

Detailed algorithmic interpretation:

- **Line 2 (Validation)**: The input message (e.g., a JSON request from a REST client or an XML document) is validated against the relevant XSD schema. This ensures mandatory fields are present, data types are correct, and cardinality constraints are satisfied. Validation failures abort processing immediately, returning a `400 Bad Request` or equivalent error. This replicates the BPEL engine's behaviour (Listing 3, line 4), where invalid input never enters the workflow.
- **Line 3 (Derive flow from BPEL)**: The BPEL file (Listing 3) is parsed to extract the sequence of operations: enrichment calls (`CRP11X1`, `CRP10X1`), the primary service invocation (`LOC3X1M`), and any conditional branching. In practice, this "derivation" is a manual translation performed once during implementation, not a runtime operation. The resulting Camel route hard-codes the sequence: `.to("direct:crp11x1").to("direct:crp10x1").to("direct:loc3x1m").`
- **Lines 4–10 (Enrichment loop)**: For each step identified in the BPEL flow, the algorithm conditionally executes enrichment services. `CRP11X1` (line 5) looks up state/province codes

Algorithm 2 Translate and Produce Deterministic Output

```
1: function TranslateAndProduce(input)
2:     validate(input, XSD)
3:     steps ← deriveFlowFromBPEL(bpel)
4:     for all step in steps do
5:         if step = CRP11X1 then
6:             input ← callCRP11X1(input)
7:         end if
8:         if step = CRP10X1 then
9:             input ← callCRP10X1(input)
10:        end if
11:        if step = LOC3X1M then
12:            mReply ← callLOC3X1M(buildMRequest(input))
13:        end if
14:    end for
15:    mapped ← applyXSLT(mReply, CHUBB/main_mapper_with_submaps.xsl)
16:    mapped ← applyBusinessRules(mapped)                    ▷ e.g., fire district
17:    writeXML(mapped, CHUBB/output/loc3x1_result.xml)
18:    return mapped
19: end function
```

when the input lacks them (e.g., inferring "CA" from a ZIP code). `CRP10X1` (line 7) performs country code enrichment. These services are invoked via SOAP (using generated client stubs from their WSDLs), and their responses augment the `input` message. This enrichment is critical: downstream services (like `LOC3X1M`) may require complete address data, and enrichment ensures it's present before invocation.

- **Line 9 (Primary service call)**: `callLOC3X1M` invokes the core location retrieval service defined in Listing 1. The `buildMRequest` function constructs a `GetLocationListRequest` object (a JAXB-generated POJO), populating it from the enriched `input`. The SOAP call returns a `GetLocationListResponse` (stored in `mReply`), which contains location data in the LOC3X1M schema format. This step corresponds to Listing 3, line 5.

- **Line 11 (XSLT mapping)**: The `mReply` (LOC3X1M format) is transformed into LOC3X1B format using an XSLT stylesheet (`main_mapper_with_submaps.xsl`). This stylesheet encodes every transformation rule from the client's `.map` files: field renaming, structural reshaping, default value application, code normalisation. XSLT is deterministic—given the same input, it produces identical output—which is essential for regression testing. The `applyXSLT` function uses a Java XSLT processor (e.g., `javax.xml.transform.Transformer`) or Camel's `.to("xslt:...)"` component.

- **Line 12 (Business rules)**: Additional business logic is applied post-transformation. For example, fire district codes may require special handling: if the `FireDistrictCode` field is empty but the address falls within a known jurisdiction, a default code is inserted. These rules are too complex or context-dependent to express in XSLT alone, so they're implemented as Java processors or Groovy scripts invoked by the Camel route.

- **Line 13 (Persist output)**: The final `mapped` XML document is serialised to `CHUBB/output/loc3x1_result.` This file serves multiple purposes: (1) it provides a human-readable audit trail, (2) it enables

regression testing (comparing current output against golden files), and (3) it can be used for downstream processing or manual review. Writing to disk replicates the BPEL engine's behaviour, where process instances often persist intermediate and final results for observability.

Step 7: Generate services for XML/JSON inputs/outputs
Implemented: XML output is produced deterministically to `CHUBB/output/loc3x1_result.xml`; REST endpoints accept JSON/XML requests and orchestrate the flow.
Enhance: Provide content negotiation for responses, validate JSON payloads against a canonical schema, and ensure both XML/JSON outputs align with the same domain model (DTOs) to avoid drift.

- **Line 14 (Return)**: The function returns the `mapped` XML, which the Camel route can optionally send to additional endpoints (e.g., a message queue, a REST endpoint, or an archival system).

Determinism and correctness: The algorithm's correctness hinges on faithful replication of legacy semantics. If the BPEL process invoked `CRP11X1` *before* `LOC3X1M`, we must do the same—reversing the order could yield incorrect results if `LOC3X1M` expects enriched data. Similarly, the XSLT transformation must implement every mapping rule exactly as specified in the `.map` files. Any deviation—omitting a field, applying the wrong default, mishandling optionals—constitutes a regression. This is why Algorithm 2 emphasises "translate" rather than "rewrite": we are mechanically converting BPEL steps and `.map` rules into equivalent Camel/XSLT constructs, preserving behaviour byte-for-byte.

Concrete Example (Real Client Assets) To ground the abstract flow in tangible artefacts, we enumerate the actual files from `CHUBB/Dependencies/` and trace their roles:

- **Contracts**: The `LocationRetrievalLOC3X1M/` directory contains `LocationRetrievalLOC3X1M.wsdl` (Listing 1), which defines the `GetLocationList` operation. Bindings specify SOAP 1.2 over HTTP, and the service endpoint URL points to a legacy system (replaced by a mock or test endpoint in the PoC). Messages reference XSD types, ensuring strong typing. Running `wsdl2java` on this WSDL generates Java classes like `GetLocationListRequest.java` and `GetLocationListResponse.java`, which our Camel route instantiates and manipulates [17].
- **Schemas**: The `Schemas/` directory houses reusable XSD files: `StatusInformation.xsd` (Listing 2), `TaxingJurisdiction.xsd` (defining tax codes and jurisdiction boundaries), `TypedAddressX3.xsd` (a complex address structure with optional fields like `AddressLine3`, `County`, `PostalCodeExtension`). These schemas are imported by multiple WSDLs, establishing a shared type library. When code generation runs, types from these schemas become Java classes annotated with JAXB bindings, enabling automatic marshalling and unmarshalling.
- **Orchestration**: `LocationRetrievalLOC3X1Process.bpel` (Listing 3) documents the workflow: receive request, enrich with `CRP11X1` and `CRP10X1`, invoke `LOC3X1M`, map to LOC3X1B, reply. In the legacy system, this BPEL process ran on Oracle SOA Suite or IBM Process Server. Our modernisation translates it into a Camel route (`src/main/resources/camel/main-route.yaml`) using Camel's YAML DSL to express the same sequence of steps [9].

- **Mapping**: `CHUBB/main_mapper_with_submaps.xsl` is a 500-line XSLT 2.0 stylesheet encoding transformations like "copy `LOC3X1M/StatusInformation/StatusCode` to `LOC3X1B/StatusInformation/`normalising codes (e.g., `SUCCESS` → `0`)," or "if `LOC3X1M/TaxingJurisdiction/FireDistrictCode` is empty, output `<FireDistrictCode>UNKNOWN</FireDistrictCode>`." This stylesheet was generated from `.map` files (Excel-like specifications maintained by business analysts), ensuring every field-level rule is auditable and version-controlled [18].

- **Execution**: A sample input file, `CHUBB/sample_input_with_submaps.xml`, provides a representative payload: an address with `City="Los Angeles"`, `StateOrProvince="CA"`, `PostalCode="90210"`, but missing `Country`. The Camel route processes this input: (1) validates it against `TypedAddressX3.xsd`, (2) invokes `CRP10X1` to enrich `Country="US"`, (3) calls `LOC3X1M` with the enriched address, receiving a response with location details and tax jurisdiction codes, (4) applies `main_mapper_with_submaps.xsl` to reshape the response into LOC3X1B format, (5) writes `CHUBB/output/loc3x1_result.xml`. This output is then compared (byte-for-byte or semantically) against a golden file to verify correctness.

Why concrete examples matter: Abstract descriptions ("we call a service and transform the response") obscure critical details. By identifying actual files—`LocationRetrievalLOC3X1M.wsdl`, `main_mapper_with_submaps.xsl`, `loc3x1_result.xml`—we make the workflow tangible. A developer can open these files, trace field mappings line-by-line, and verify that the Camel route implements the intended behaviour. This concreteness is essential for debugging ("the output is missing `StatusMessage`; let's check if the XSLT handles `minOccurs="0"` correctly") and for onboarding new team members ("read `LocationRetrievalLOC3X1Process.bpel` to understand the workflow, then examine `main-route.yaml` to see how we translated it").

1.4 Modernisation Objectives

Having understood the legacy artefacts (Section 1.2) and their integration (Section 1.3), we now articulate the modernisation goals. These objectives guide all technical decisions and serve as acceptance criteria for the PoC.

- **Replace legacy orchestration (BPEL, proprietary mappers) with Spring Boot + Apache Camel** [3]: The client's existing system relies on vendor-specific BPEL engines (Oracle SOA Suite, IBM WebSphere Process Server) and proprietary mapping tools. These platforms incur licensing costs, require specialised skills, and resist modern DevOps practices (e.g., Git-based version control, Docker containerisation, CI/CD pipelines). By re-implementing orchestration in Apache Camel—a vendor-neutral, open-source integration framework—we eliminate vendor lock-in. Camel routes are expressed in YAML or Java DSL, making them readable, testable, and versionable. Spring Boot provides the application runtime, offering autoconfiguration, embedded servers (Tomcat, Jetty), and production-ready features (health checks, metrics, externalised configuration). This combination is cloud-native: the PoC can be packaged as a Docker image, deployed to Kubernetes, and scaled horizontally.

- **Implement data mapping using XSLT and lightweight processors** [18]: The client's `.map` files were authored in proprietary mapping tools (drag-and-drop graphical interfaces). These tools generate artefacts (e.g., Oracle Service Bus `.xqy` files, IBM `.mxl` files) that are opaque, difficult to version, and impossible to unit-test in isolation. By translating `.map` rules into XSLT stylesheets, we gain portability (XSLT processors are ubiquitous), testability (XSLT transformations can be unit-tested with XML input/output fixtures), and transparency (developers can read and modify `.xsl` files). For transformations too complex for

XSLT (e.g., invoking external APIs, applying stateful business rules), we use lightweight Java processors or Groovy scripts within the Camel route. This hybrid approach balances declarative simplicity (XSLT) with procedural expressiveness (Java/Groovy).

- **Integrate enrichment (state/country lookups) and external services using HTTP/-SOAP clients** [16, 17]: The legacy workflow invokes multiple external services (`CRP11X1` for state enrichment, `CRP10X1` for country enrichment, `LOC3X1M` for location retrieval). These services expose SOAP interfaces defined by WSDLs in `Dependencies/`. Our modernisation uses Apache CXF (a JAX-WS implementation) to generate type-safe client stubs from these WSDLs. Camel's `cxf` component integrates these stubs seamlessly, allowing routes to invoke SOAP services with a single `.to("cxf:bean:loc3x1mClient")` statement. For services transitioning to REST, Camel's `http` component provides equivalent functionality. This approach preserves integration contracts (WSDL-defined message formats) whilst enabling gradual migration to modern protocols.
- **Deliver a PoC that writes a verifiable XML artefact to `CHUBB/output/loc3x1_result.xml`**: The PoC's success criterion is producing deterministic, correct output. For every sample input (e.g., `sample_input_with_submaps.xml`), the Camel route must generate an XML file that (1) validates against the LOC3X1B schema, (2) contains all mandatory fields populated correctly, (3) matches the expected output (golden file) byte-for-byte or semantically (ignoring whitespace/comments). This output serves as a regression test baseline: any code change must continue producing identical output for the same input. Writing to a file (`loc3x1_result.xml`) provides an audit trail and simplifies comparison (tools like `xmldiff` or custom validators can assert equivalence). In production, this output would be sent to downstream systems (message queues, databases, REST endpoints), but file-based validation is sufficient for the PoC.

Why these objectives? They address the client's pain points: high licensing costs (vendor lock-in), inflexible deployment (monolithic platforms), opaque logic (proprietary mappers), and difficult testing (no CI/CD integration). By targeting open-source, cloud-native technologies, we enable the client to deploy on any infrastructure (AWS, Azure, on-premises Kubernetes), reduce operational costs (no vendor licences), and accelerate development (Git workflows, automated testing, faster iteration). The PoC demonstrates feasibility: "Can we re-implement this workflow in Camel without changing behaviour?" If the PoC succeeds, the client gains confidence to modernise additional workflows using the same approach.

2 Technology Primer

2.1 Java and Maven

Java runs on the JVM and compiles source files into bytecode. Maven is a build tool that manages dependencies, compilation, packaging, and plugins [5]. The project uses Maven goals like `compile`, `package`, and `spring-boot:run`.

2.2 Spring Boot

Spring Boot simplifies building production-ready services with embedded servers, autoconfiguration, and actuator endpoints [10]. It provides a clean entry point (`mainClass`) and integrates seamlessly with Camel.

2.3 Apache Camel and YAML DSL

Apache Camel is an integration framework implementing Enterprise Integration Patterns (EIP). Routes define message flow using components (HTTP, timer, log, etc.). This project uses the YAML DSL to declare routes in `.yaml` files loaded from the classpath [3].

2.4 SOAP, WSDL, JAXB, and JAX-WS

SOAP is an XML-based messaging protocol; WSDL describes service contracts and bindings [16, 17]. JAXB maps XML schemas (XSD) to Java classes (POJOs), while JAX-WS provides SOAP client/server APIs [6, 7]. Tools like `xjc` and `wsimport`, or Apache CXF [4], generate code from XSD/WSDL.

2.5 BPEL

BPEL (Business Process Execution Language) defines orchestrations of service invocations in XML [9]. The client's BPEL file documents the legacy flow; Camel routes replace its orchestration role in the PoC.

2.6 XSLT

XSLT transforms XML using declarative templates [18]. The mapping logic is encoded in `CHUBB/main_mapper_with` mirroring rules from the client's `.map` files.

2.7 Supporting Libraries

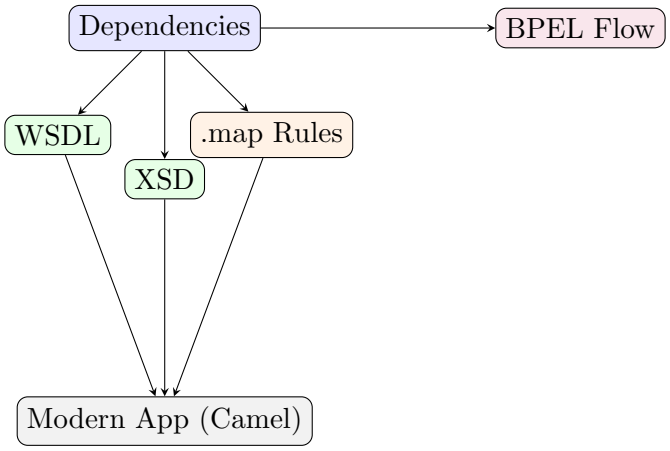
MapStruct (compile-time mappers) [12], Lombok (boilerplate reduction) [11], Jackson XML (XML binding) [2]. These improve productivity without changing core logic.

3 Client `Dependencies` Folder

3.1 Contents

- Service Contracts: `LocationRetrievalLOC3X1*/` WSDL/XSD; `CRP11X1`, `CRP10X1` WSDL/XSD.
- Domain Schemas: `Schemas/` (e.g., `StatusInformation.xsd`, `TaxingJurisdiction.xsd`).
- Mapping Specs: `LOC3X1/*.map` defining request/reply transformations.
- Process Doc: `LocationRetrievalLOC3X1Process.bpel` legacy orchestration.

3.2 Relationship Diagram



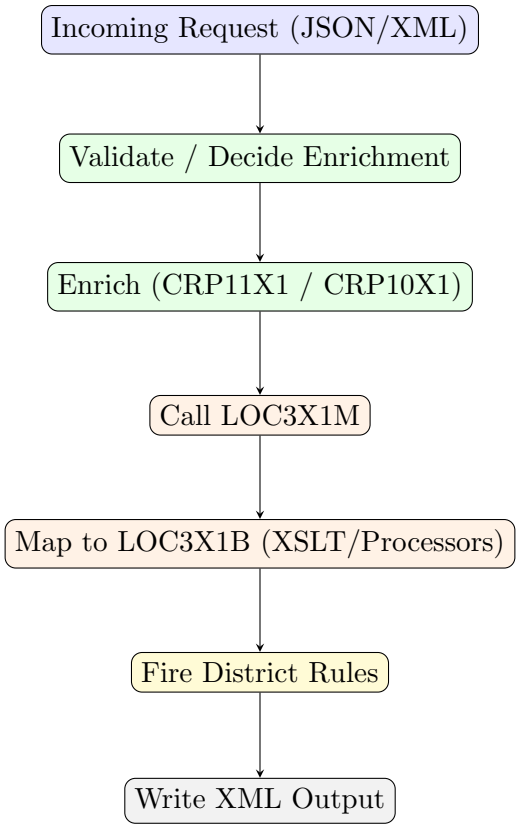
4 Solution Architecture

4.1 Repository Layout

- `src/main/java`: controllers, services, processors.
- `src/main/resources`: `application.yml`, Camel routes (YAML), templates.
- `CHUBB/`: XSLT, sample input, output folder, Dependencies (specs).
- `libs/`: external JARs (POJOs, SOAP clients).

4.2 Orchestration Flow

Camel routes implement validation, enrichment, external calls, and mapping. A simplified flow is shown below.



4.3 Key Components

- **Validation Service**: decides if CRP11X1/CRP10X1 lookups are needed.
- **Mapping Service**: orchestrates parsing and XSLT transformation; shapes LOC3X1M reply to LOC3X1B.
- **Processors**: build requests, apply rules (e.g., fire district), convert data structures.
- **XSLT**: implements field-level mapping rules derived from `.map` specs.

5 Data Mapping: Tables and Pseudocode

5.1 Representative Field Mapping

Source (RAND/LOC3X1M)	Target (LOC3X1B)	Rule
`StatusInformation/StatusCode`	`StatusInformation/StatusCode`	Copy, normalize codes
`TaxingJurisdiction/FireDistrictCode`	`PublicProtection/FireDistrictCode`	Copy, apply defaulting
`Address/StateOrProvince`	`Address/StateOrProvince`	Enriched via CRP11X1 when missing
`Address/Country`	`Address/Country`	Enriched via CRP10X1 when missing
`Location/Coordinates`	`Location/Coordinates`	Copy; validate range

5.2 Pseudocode: Transform Flow

Algorithm 3 High-level Transform and Orchestration

```
1: function ProcessRequest(input)
2:     validate(input)
3:     if needsStateEnrichment(input) then
4:         input ← callCRP11X1(input)
5:     end if
6:     if needsCountryEnrichment(input) then
7:         input ← callCRP10X1(input)
8:     end if
9:     loc3x1mReply ← callLOC3X1M(buildRequest(input))
10:    mapped ← applyXSLT(loc3x1mReply, main_mapper_with_submaps)
11:    mapped ← applyFireDistrictRules(mapped)
12:    writeXML(mapped, CHUBB/output/loc3x1_result.xml)
13:    return mapped
14: end function
```

6 Build and Run

6.1 Configuration

- `application.yml`: enables YAML routes from `classpath:camel/*.yaml, classpath:*.yaml`; defines service host/port placeholders.
- External JARs in `libs/`: POJOs and SOAP clients referenced via Maven system scope.

6.2 Commands

- Development run: `mvn -q -DskipTests -Dspring-boot.run.arguments=--server.port=8081 spring-boot:run`
- Packaging (optional): `mvn package`

6.3 Verification

- Demo endpoint triggers the XSLT transformation and writes the output file to `CHUBB/output/loc3x1_result`
- Inspect the file to confirm size and content against expectations (e.g., `happy_path_example.xml`).

7 Glossary

- **WSDL**: XML description of SOAP services (operations, bindings).
- **XSD**: XML Schema Definition, types and constraints.
- **SOAP**: XML-based messaging protocol, often over HTTP.
- **BPEL**: XML language for service orchestration.
- **Camel Route**: Directed flow of message processing steps.
- **XSLT**: Declarative transformation language for XML.
- **POJO**: Plain Old Java Object, simple data holder.

8 Agent-Based Automation with Python, Ollama, and LangGraph

8.1 Objective and Scope

This section evaluates replacing the Java/Camel orchestration with a Python implementation that preserves deterministic mapping while introducing deep-agent capabilities for assistive tasks. The proposed stack uses a local LLM via Ollama [13], a graph-based workflow engine via LangGraph [1], and a FastAPI service layer [14]. Deterministic XML transforms remain in XSLT using `lxml` [8], and SOAP integration is handled by `zeep` [15].

8.2 Feasibility and Constraints

- **Realizable**: All current stages (validate, enrich, external call, map, rules, persist) have Python equivalents.
- **Determinism**: Field-level transforms remain in XSLT; agents provide non-authoritative assistance (QA, normalization, suggestions).
- **Parities**: A golden-file test suite must prove Python outputs equal to the Java implementation for representative inputs.

8.3 Architecture Overview

- **Service Layer**: FastAPI endpoints mirror current REST surfaces.
- **Workflow**: LangGraph nodes for validation, enrichment (CRP11X1/CRP10X1), LOC3X1M call, XSLT mapping, rule application, persistence.
- **Agents**: Ollama-backed nodes augment input hygiene and anomaly detection, logging explainable recommendations without mutating canonical data.
- **Integration**: `zeep` consumes WSDLs from `Dependencies` to invoke SOAP services; `lxml.etree.XSLT` runs `main_mapper_with_submaps.xsl`.

8.4 Workflow (Pseudocode)

Algorithm 4 Python Orchestration with Deterministic Mapping

```
function ProcessRequestPython(input)
    validate(input)                                              ▷ deterministic checks
    if needsStateEnrichment(input) then
        input ← callCRP11X1(input)                               ▷ zeep/WSDL
    end if
    if needsCountryEnrichment(input) then
        input ← callCRP10X1(input)                               ▷ zeep/WSDL
    end if
    loc3x1mReply ← callLOC3X1M(buildRequest(input))
    mapped ← applyXSLT(loc3x1mReply, main_mapper_with_submaps)   ▷ lxml.XSLT
    mapped ← applyFireDistrictRules(mapped)                      ▷ explicit rules
    agentQA ← ollamaQA(mapped)                                   ▷ non-authoritative, logged
    writeXML(mapped, CHUBB/output/loc3x1_result.xml)
    return mapped
end function
```

8.5 Migration Plan

- **Baseline**: Export current Java outputs for multiple inputs as golden files.
- **Deterministic Core**: Implement FastAPI + LangGraph nodes; wire zeep calls and XSLT mapping first.
- **Agent Augmentation**: Add Ollama nodes for QA and normalization that produce logs and operator guidance.
- **Parity Tests**: Require byte-equivalent or schema-equivalent outputs against golden files.
- **Rollout**: Expose identical endpoints; run side-by-side until confidence is achieved.

8.6 Pros and Cons

- **Pros**: Deterministic transforms preserved; local LLM (privacy); graph workflows are testable and maintainable.
- **Cons**: Added effort to reimplement SOAP bindings; LLM outputs are non-deterministic and must be scoped to assistive roles.

8.7 Conclusion

A Python + LangGraph + Ollama approach is feasible when deterministic mapping stays in XSLT, agents are constrained to advisory functions, and parity testing governs acceptance. This yields a maintainable, auditable system with optional intelligent QA.

9 References

References

[1] LangChain AI. Langgraph documentation. https://langchain-ai.github.io/langgraph/. Accessed 2025-11.

[2] FasterXML. Jackson xml module. https://github.com/FasterXML/jackson-dataformat-xml. Accessed 2025-11.

[3] Apache Software Foundation. Apache camel documentation. https://camel.apache.org/manual/latest/. Accessed 2025-11.

[4] Apache Software Foundation. Apache cxf documentation. https://cxf.apache.org/docs/. Accessed 2025-11.

[5] Apache Software Foundation. Apache maven project. https://maven.apache.org/guides/. Accessed 2025-11.

[6] Eclipse Foundation. Jakarta xml binding (jaxb) specification. https://projects.eclipse.org/projects/ee4j.jaxb/specification. Accessed 2025-11.

[7] Eclipse Foundation. Jakarta xml web services (jax-ws). https://jakarta.ee/specifications/xml-web-services/. Accessed 2025-11.

[8] lxml Project. lxml: Xml and html with python. https://lxml.de/. Accessed 2025-11.

[9] OASIS. Oasis bpel 2.0 specification. https://docs.oasis-open.org/wsbpel/2.0/wsbpel-v2.0.html. Accessed 2025-11.

[10] Pivotal/VMware. Spring boot reference documentation. https://docs.spring.io/spring-boot/docs/current/reference/html/. Accessed 2025-11.

[11] Lombok Team. Project lombok documentation. https://projectlombok.org/. Accessed 2025-11.

[12] MapStruct Team. Mapstruct reference guide. https://mapstruct.org/documentation/stable/reference/html/. Accessed 2025-11.

[13] Ollama Team. Ollama documentation. https://ollama.com/docs. Accessed 2025-11.

[14] Tiangolo. Fastapi documentation. https://fastapi.tiangolo.com/. Accessed 2025-11.

[15] Michael van Tessel and Contributors. Zeep soap client documentation. https://docs.python-zeep.org/en/master/. Accessed 2025-11.

[16] W3C. Soap 1.2 (w3c recommendation). https://www.w3.org/TR/soap12-part1/. Accessed 2025-11.

[17] W3C. Web services description language (wsdl) 1.1. https://www.w3.org/TR/wsdl. Accessed 2025-11.

[18] W3C. Xsl transformations (xslt) 1.0/2.0. https://www.w3.org/TR/xslt-30/. Accessed 2025-11.